APT

ATYPICAL TECHNOLOGIES PVT LTD

Interview Assignment Submission

Real-time Database Change Notification System

Submitted by	Omkar Patel
Role Applied	Backend Developer — APT
Date	May 2026
GitHub	github.com/Omkaarrr
Tech Stack	Node.js · PostgreSQL · WebSockets
Assignment	DB Changes → Real-time Client Notifications

1. Overview

This report documents the design and implementation of a real-time database notification system built for the APT backend interview assignment. The system pushes database changes to connected clients instantly, without any client-side polling.

2. Problem Statement

The assignment requires a backend service that:

- Monitors an orders table for any INSERT, UPDATE, or DELETE operations.
- Notifies all connected clients immediately whenever data changes.
- Does not rely on polling — clients should receive a push, not ask repeatedly.
- Includes a client (browser, CLI, or script) that displays live updates.

The exact schema specified in the assignment:

```
CREATE TABLE orders (
    id            SERIAL PRIMARY KEY,
    customer_name VARCHAR(255)  NOT NULL,
    product_name  VARCHAR(255)  NOT NULL,
    status        VARCHAR(50)   NOT NULL DEFAULT 'pending'
                  CHECK (status IN ('pending', 'shipped', 'delivered')),
    updated_at    TIMESTAMP     NOT NULL DEFAULT NOW()
);
```

3. Approach & Design Decisions

3.1 Why PostgreSQL LISTEN / NOTIFY?

Three approaches were evaluated:

Option	Mechanism	Decision
Polling	Client requests DB every N seconds	Rejected — wasteful
CDC / Debezium	Reads Postgres WAL, streams via Kafka	Rejected — too heavy
LISTEN / NOTIFY	Built-in Postgres trigger mechanism	**Chosen**

3.2 Why WebSockets over SSE or Long-Polling?

- WebSockets — persistent full-duplex connection; ideal for continuous real-time push.

- SSE — one-way only; less flexible for future bidirectional needs.
- Long-polling — a workaround, not a proper solution.

3.3 Data Flow

A DB mutation triggers the following chain automatically:

```
DB change  →  Postgres trigger fires
           →  pg_notify('orders_channel', JSON payload)
           →  Node.js listenerClient receives 'notification' event
           →  broadcast() pushes JSON to all WebSocket clients
           →  Browser UI re-renders in real-time
```

4. Tech Stack

Layer	Choice	Reason
Runtime	Node.js v18+	Non-blocking I/O; ideal for real-time
Framework	Express	Minimal, standard, well-supported
Database	PostgreSQL	Native LISTEN/NOTIFY support
DB Client	node-postgres (pg)	Reliable; supports dedicated listener client
Real-time	ws library	Lightweight WebSocket, no socket.io overhead
Client	Vanilla HTML/JS	No framework needed; fast, dependency-free

5. Project Structure

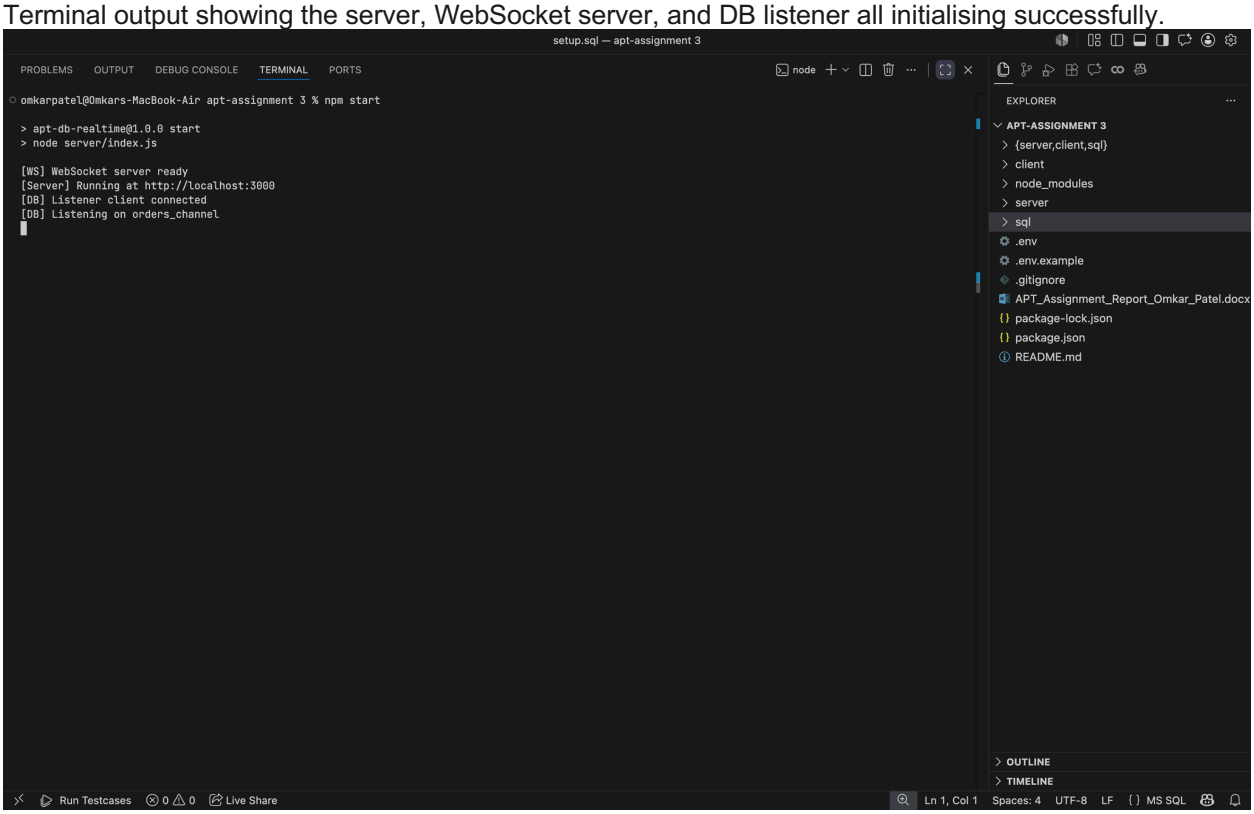
```
apt-assignment/
├── server/
│   ├── index.js       Entry point — boots HTTP, WebSocket, DB listener
│   ├── db.js          Pool (queries) + dedicated listener client
│   ├── websocket.js   WS server setup + broadcast() helper
│   └── routes.js      REST API — GET / POST / PATCH / DELETE /api/orders
├── client/
│   └── index.html     Browser client — live event feed + orders table
├── sql/
│   └── setup.sql      Table schema + trigger function + seed data
```

```
├── .env.example
├── package.json
└── README.md
```

6. Output Screenshots

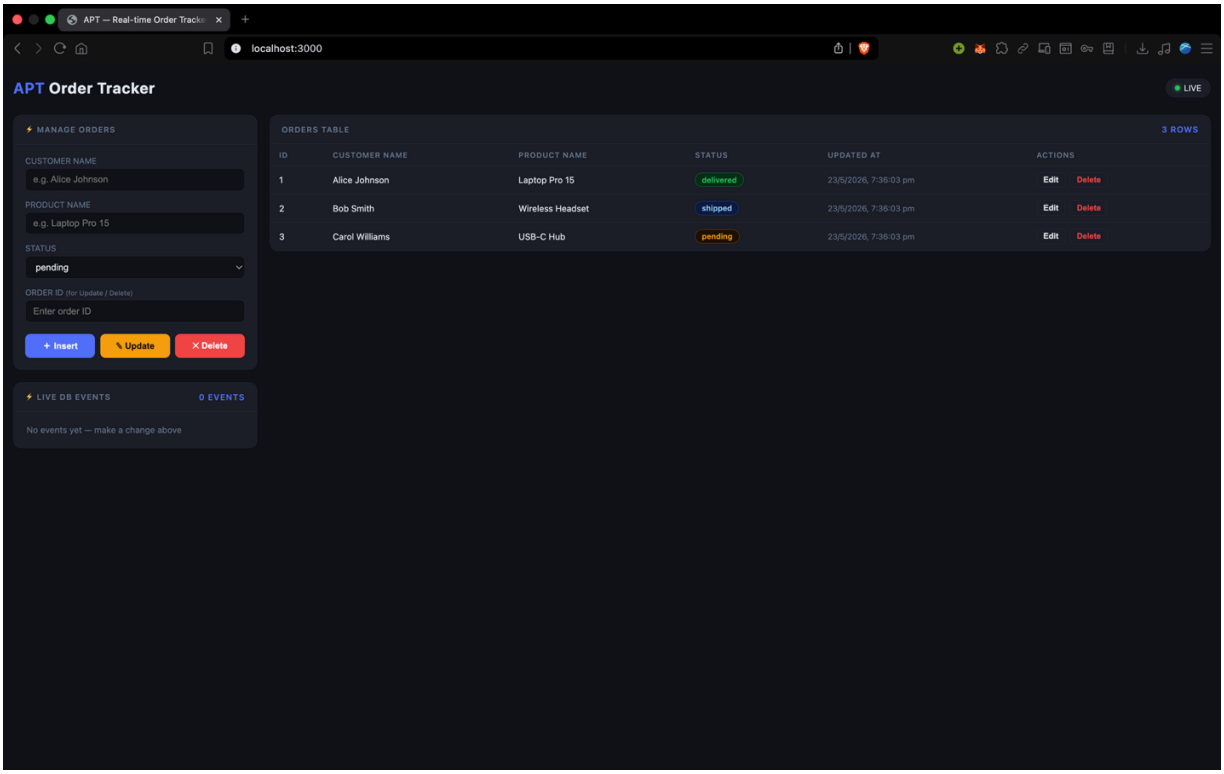
6.1 Server Starting Up

Terminal output showing the server, WebSocket server, and DB listener all initialising successfully.



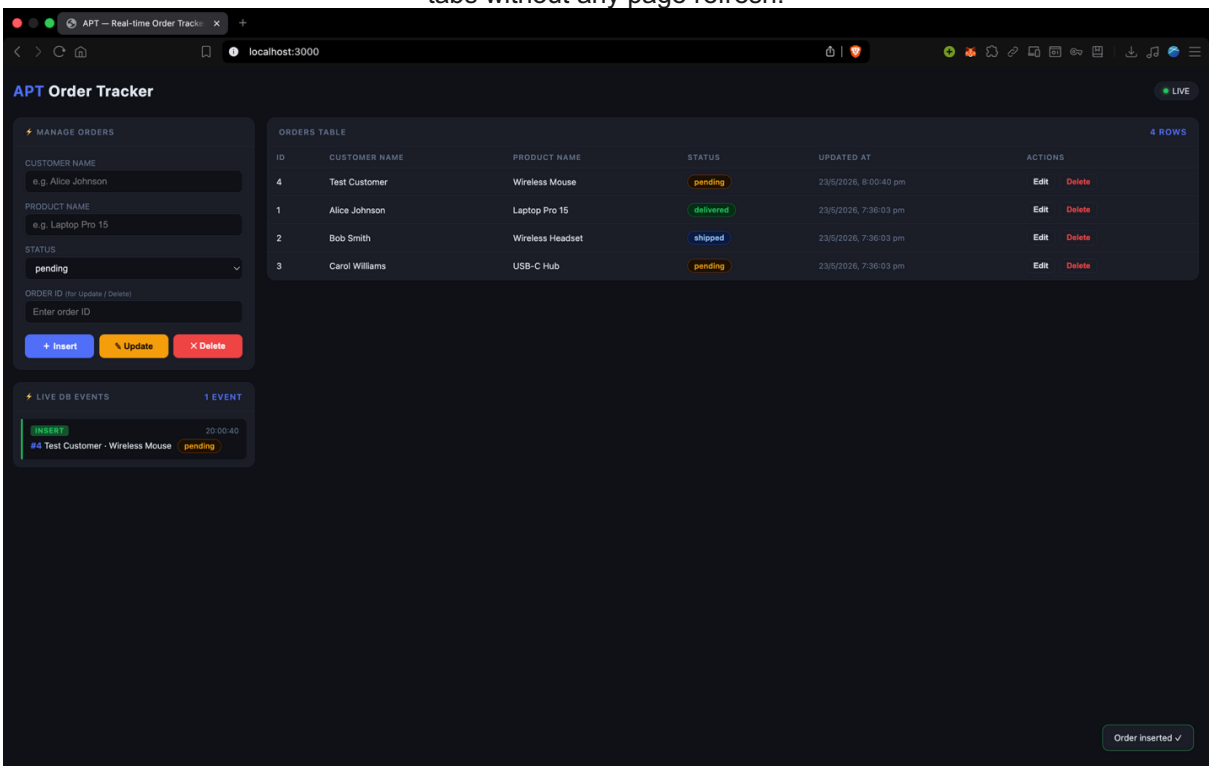
6.2 Browser Client — Initial State

Browser at localhost:3000 showing the orders table loaded with seeded data and the LIVE WebSocket indicator active.



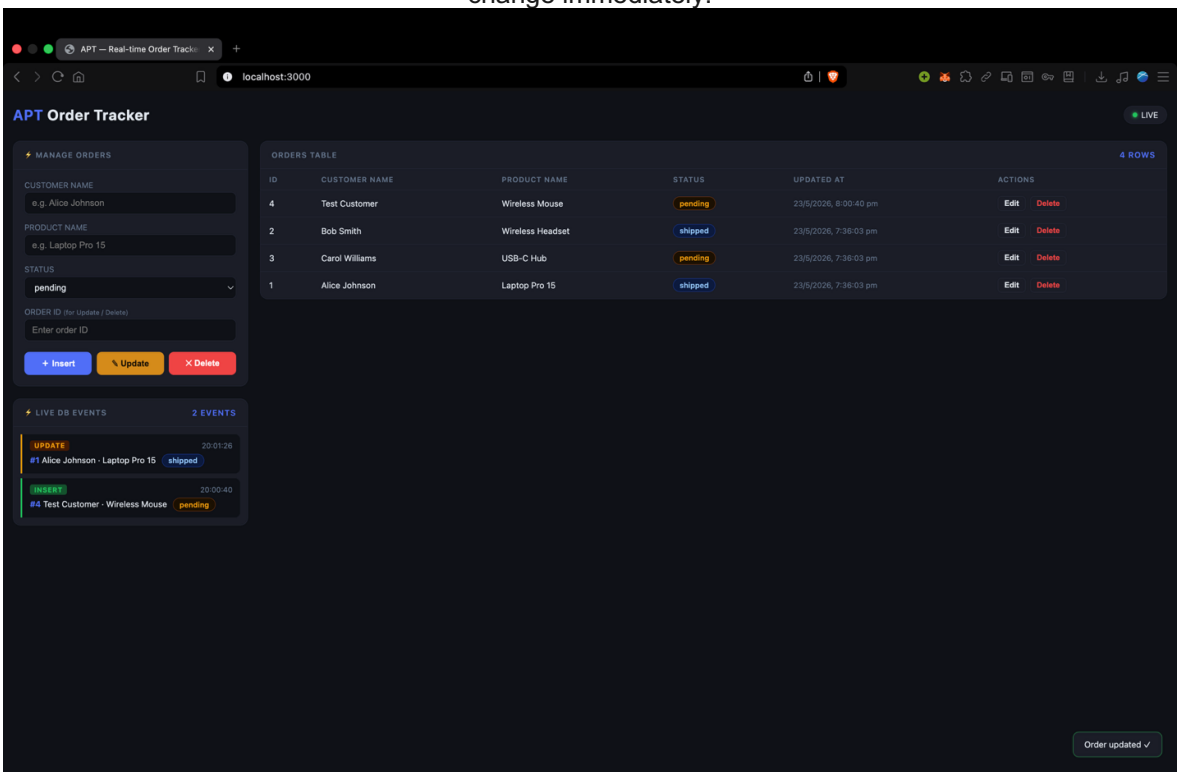
6.3 INSERT Event — Real-time Notification

A new order is inserted via the form (or curl). The Live DB Events feed updates instantly across all open tabs without any page refresh.



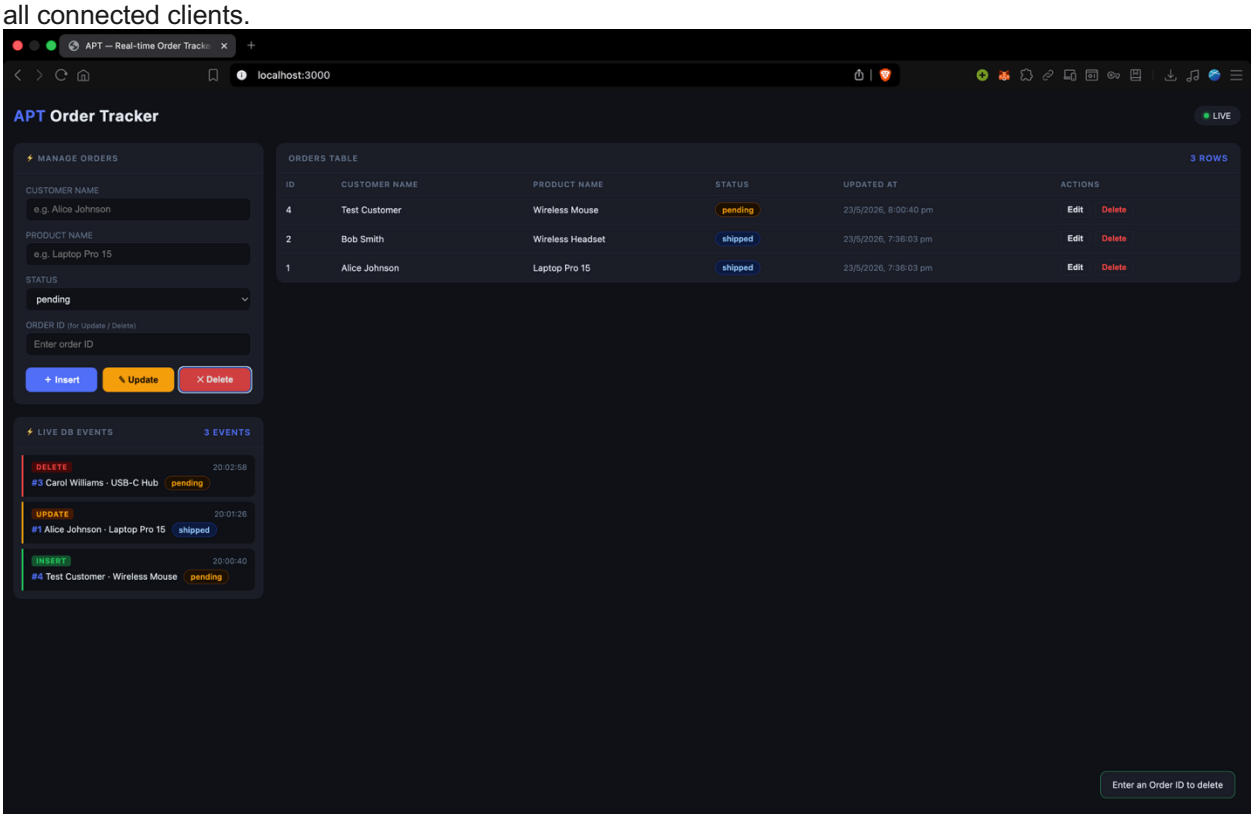
6.4 UPDATE Event — Status Change

An existing order's status is patched to 'shipped'. The event feed and the orders table both reflect the change immediately.



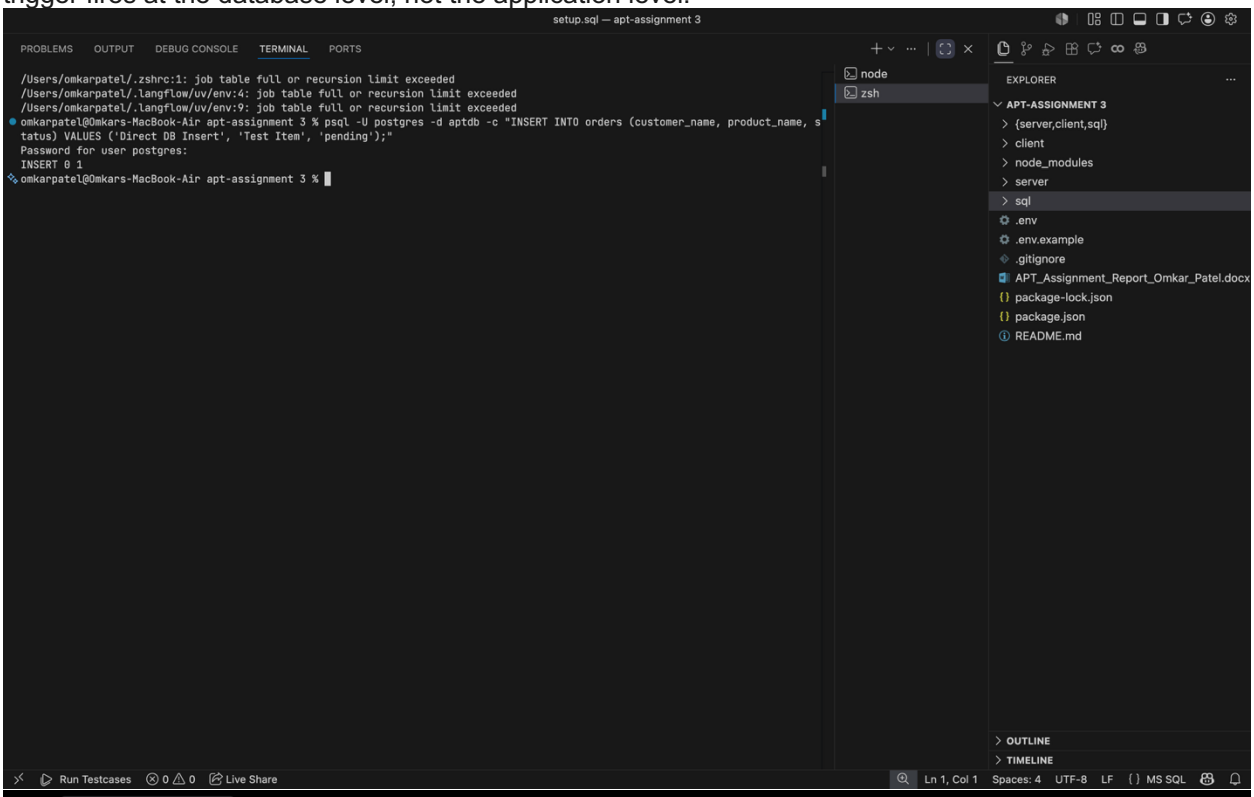
6.5 DELETE Event

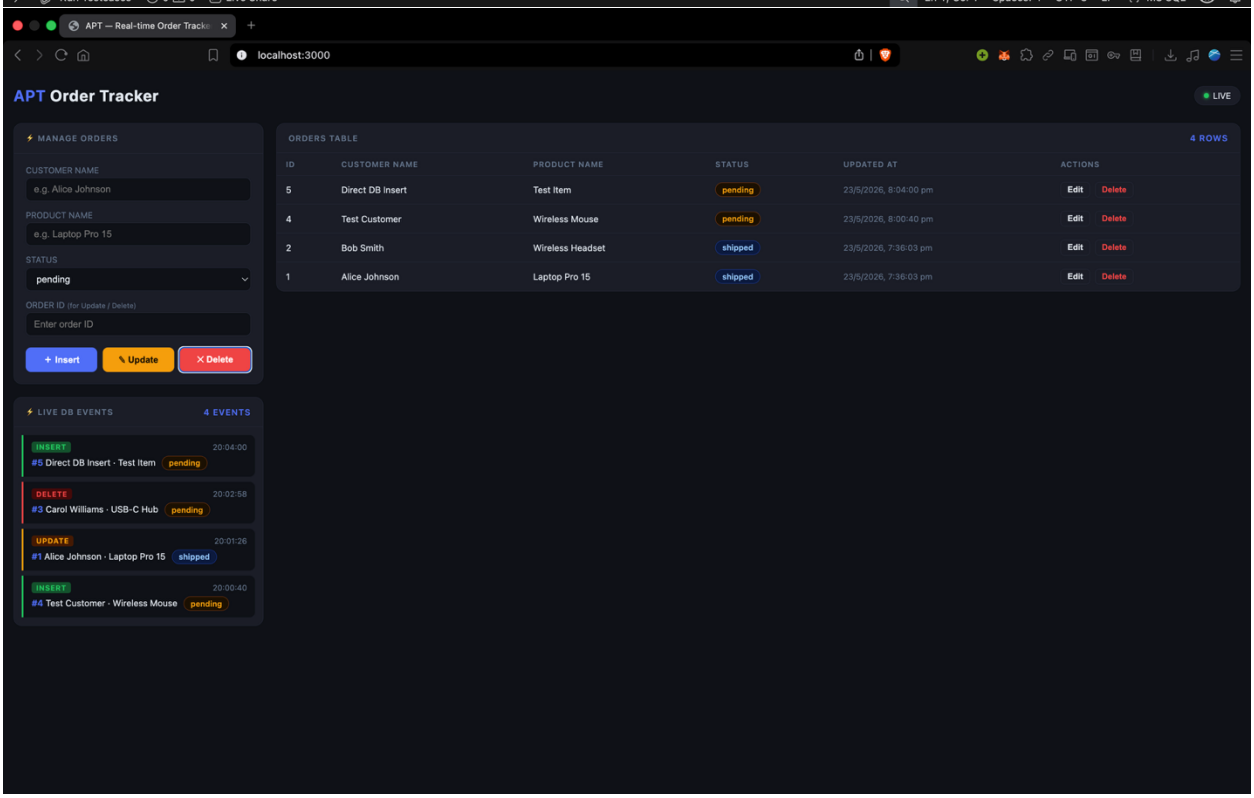
An order is deleted. The row disappears from the table and a DELETE event appears in the feed across all connected clients.



6.6 Direct psql Mutation (No API)

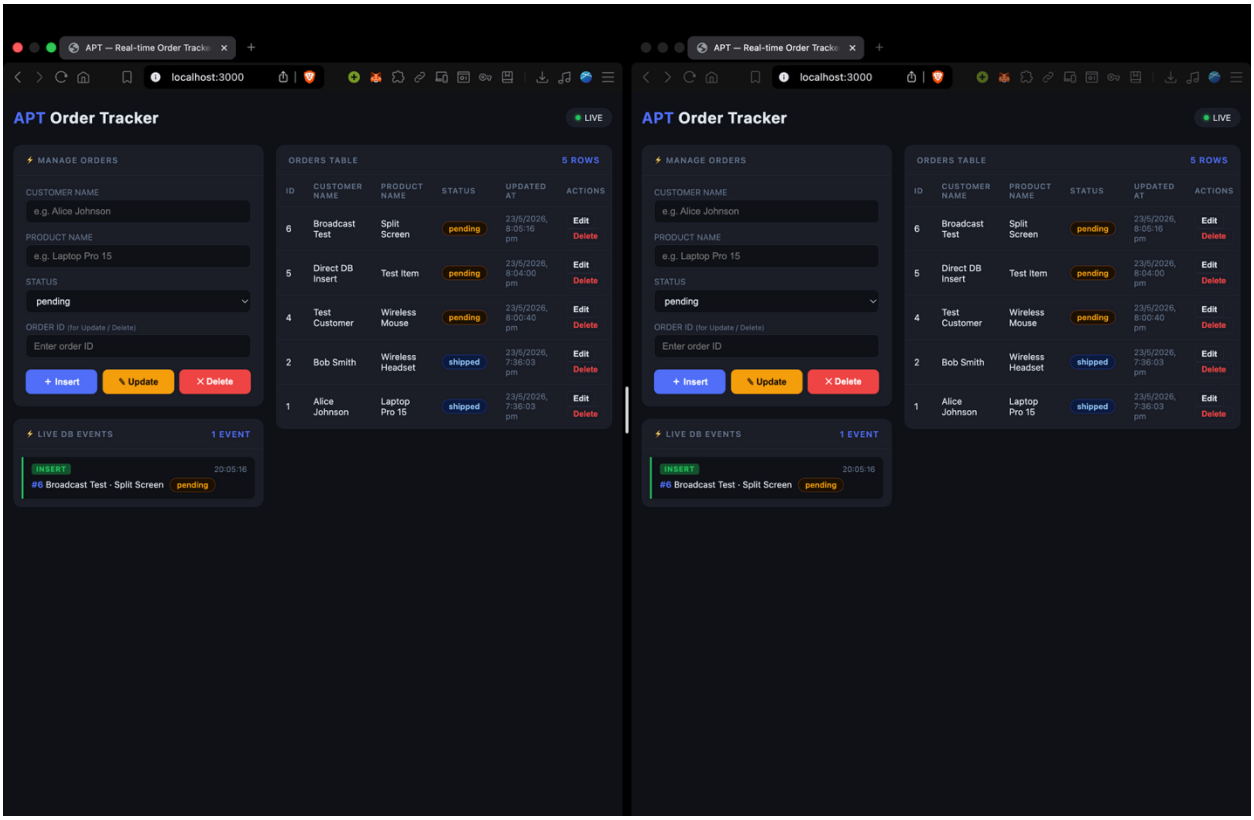
Running an INSERT directly in psql (bypassing the REST API) still triggers the notification, proving the trigger fires at the database level, not the application level.





6.7 Multiple Tabs — Simultaneous Update

Two browser windows open side-by-side. A single change in one triggers both to update simultaneously.



7. Evaluation Criteria Checklist

Criteria	How it is Addressed	Status
Design Thinking	Three approaches compared (polling, CDC, LISTEN/NOTIFY). Dedicated listener client vs pool explained. Scalability section in README covers Redis pub/sub for multi-instance.	**Addressed**
Correctness	Trigger fires on INSERT, UPDATE, DELETE — including direct psql mutations. All connected clients receive push within milliseconds.	**Addressed**
Code Quality	Concerns separated across four modules (db, websocket, routes, index). Inline comments explain non-obvious decisions. Full CRUD with input validation.	**Addressed**
Documentation	README includes: setup steps, curl examples, psql test, REST API table, WebSocket message format, scalability notes, and a data-flow diagram.	**Addressed**

8. How to Run

```
# 1. Create the database
psql -U postgres -c "CREATE DATABASE aptdb;"

# 2. Run schema + trigger + seed data
psql -U postgres -d aptdb -f sql/setup.sql

# 3. Install dependencies
npm install

# 4. Configure environment
cp .env.example .env   # edit DB credentials

# 5. Start the server
npm start

# 6. Open the browser client
open http://localhost:3000
```

Open multiple browser tabs to see all of them update simultaneously when any change is made — via the UI, curl, or direct psql.

Thank you for the opportunity.

Omkar Patel · omkarpatelhere@gmail.com · GitHub